Adopter Portal API Reference

This document defines the adopter-facing endpoints delivered in Sprint 3: onboarding and profile management, account closure, government ID verification, adoption applications, visit scheduling, vet appointment viewing, favorites, and the adopted-pet health passport view. It follows the same conventions established in 07-Public API Reference — see that document for base URL, standard response structure, and error code definitions, which are not repeated here.

1. Profile Endpoints

1.1 GET /api/v1/adopters/me

Returns the full profile of the currently logged-in adopter, including all schema fields from the ADOPTER table except those explicitly excluded for security.

Request

Field	Value
Method	GET
Auth required	Yes — Adopter only
Path/Body fields	None — adopter identified via req.user.userID

Response

```
200 OK
{
  "success": true,
  "message": "Profile retrieved successfully",
  "data": {
    "adopterID": 12,
    "adopterName": "...",
    "adopterEmail": "...",
    "avatarSeed": "a1b2c3d4-...",
    "housingType": "...",
    "ownsOrRents": "...",
    "householdSize": 3,
    "...": "...all other ADOPTER fields except excluded ones"
  }
}
```

Error Cases

Scenario	Error Code	HTTP Status
No/invalid token	UNAUTHORIZED	401
Adopter record does not exist	NOT_FOUND	404

> Excluded fields: adopterPassword and stripeCustomerID are never included in this response, regardless of role — this mirrors the NF-10 data privacy requirement. avatarSeed IS included — it is not sensitive, and the frontend needs it to render the adopter's avatar via the shared DiceBear config (see Avatar Picker spec).

1.2 PUT /api/v1/adopters/me

Allows the logged-in adopter to update their profile fields. Accepts a partial body — only fields present in the request are updated. Password changes are out of scope for this endpoint.

Request

Field	Value
Method	PUT
Auth required	Yes — Adopter only
Body fields	Any subset of updatable ADOPTER fields (see design note)

> *Enum fields are validated against their allowed values before the update is applied: housingType, ownsOrRents, employmentStatus, activityLevel, preferredSize, preferredAgeRange, adopterType. An invalid enum value returns VALIDATION_ERROR before any write occurs.*

Response

```
200 OK
{
  "success": true,
  "message": "Profile updated successfully",
  "data": { "...": "same shape as GET /adopters/me" }
}
```

Error Cases

Scenario	Error Code	HTTP Status
No/invalid token	UNAUTHORIZED	401
Invalid enum value in body	VALIDATION_ERROR	422
Attempt to update a disallowed field	VALIDATION_ERROR	422
Adopter record does not exist	NOT_FOUND	404

> Disallowed fields: adopterEmail, adopterPassword, adopterRiskFlag, preQualifyFlag, and accountStatus cannot be updated through this endpoint — these are either admin-controlled, security-sensitive, or reserved for the AI matcher (Sprint 6). Any of these present in the request body are silently ignored, not written. avatarSeed IS an allowed field — a string, up to 64 characters, no additional validation beyond standard length/type checking (any string is valid input to the DiceBear generator).

> Matcher cache: if the request body includes any of the fields the compatibility matcher scores on (numChildren, currentPets, openToSpecialNeeds, preferredSize, preferredAgeRange, preferredBreedID, housingType, yardAvailable, petExperience, activityLevel), the adopter's cached matches are deleted in the same transaction as the profile update, so

> the next GET /api/v1/adopters/me/matches recomputes them. Updates that touch none of these fields leave the matches cached.

1.3 PATCH /api/v1/adopters/me/onboarding-step

Advances the adopter through the multi-step onboarding wizard.

Request

Field	Value
Method	PATCH
Auth required	Yes — Adopter only
Body fields	step (int, required, 2-7)

> *Advance-only: the server computes nextStep = min(max(currentStep, step + 1), 7) — a lower or equal step value in the body is ignored rather than moving the adopter backward, and step is capped at 7.*

Response

200 OK, same full profile shape as GET /adopters/me (1.1).

Error Cases

Scenario	Error Code	HTTP Status
No/invalid token	UNAUTHORIZED	401
step missing or not an integer 2-7	BAD_REQUEST	400

1.4 PATCH /api/v1/adopters/me/onboarding-complete

Marks onboarding complete once all required fields are present.

Request

Field	Value
Method	PATCH
Auth required	Yes — Adopter only
Body fields	none

> *Required before completion: adopterDOB, adopterSex, adopterPhone, addressLine1, city, state, zip, country, housingType, ownsOrRents, householdSize, numChildren, employmentStatus, activityLevel, petExperience, and a submitted Government ID (see Section 2). Adoption-preference fields (Step 6) are intentionally not required for completion.*

Response

200 OK, sets onboardingComplete: true and onboardingStep: 7, returns same full profile shape as GET /adopters/me (1.1).

Error Cases

Scenario	Error Code	HTTP Status

No/invalid token	UNAUTHORIZED	401
One or more required fields still missing	CONFLICT	409 (message lists which fields)

1.5 DELETE /api/v1/adopters/me

Closes the adopter's account, either by deactivating it (data retained) or permanently deleting it.

Request

Field	Value
Method	DELETE
Auth required	Yes — Adopter only
Body fields	mode (string, required — "deactivate" or "delete")

> *deactivate sets accountStatus to Deactivated and clears the refresh token (soft, reversible by support). delete removes the adopter's government ID, favorites, visits, and adoption applications, then the adopter and USERS rows themselves, and deletes the stored government-ID file from Storage. Both modes force a logout. Either mode is blocked while the adopter has an Accepted (active) adoption application on record.*

Response

```
200 OK
{
  "success": true,
  "message": "Account deleted",
  "data": null
}
```

(or "Account deactivated" depending on mode)

Error Cases

Scenario	Error Code	HTTP Status
No/invalid token	UNAUTHORIZED	401
mode missing or not one of the two allowed values	VALIDATION_ERROR	422
Adopter has an active (Accepted) adoption application	CONFLICT	409

2. Government ID Verification

These endpoints handle identity document upload and status retrieval for adoption-critical identity verification (Requirement A-01). Files are stored in Supabase Storage; only metadata is persisted in the GOVERNMENT_ID table.

2.1 POST /api/v1/adopters/me/government-id

Uploads a government ID document for identity verification. Stores file metadata in the GOVERNMENT_ID table; the file itself is uploaded to the government-ids/ bucket in Supabase Storage.

Request

Field	Value
Method	POST
Auth required	Yes — Adopter only
Content-Type	multipart/form-data
Body fields	idType (string, required), idNumber (string, required, ≤45 chars), file (JPEG/PNG/WebP/HEIC/PDF, ≤5 MB, required)

Response

```
201 Created
{
  "success": true,
  "message": "Government ID submitted successfully",
  "data": {
    "governmentIDID": 7,
    "userID": 12,
    "userType": "Adopter",
    "idType": "Driver's License",
    "idNumber": "*****4567",
    "documentURL": "...",
    "verificationStatus": "Pending"
  }
}
```

Error Cases

Scenario	Error Code	HTTP Status
No/invalid token	UNAUTHORIZED	401
Missing idType, idNumber, or file	VALIDATION_ERROR	422
File exceeds 5 MB or wrong type	BAD_REQUEST	400
Government ID already submitted for this adopter	CONFLICT	409

> *Response masking: idNumber is masked in the response (e.g. *****4567) — this is a deliberate current design choice, not a placeholder for a future fix. Revisit only if a product requirement emerges for the adopter to view their full submitted ID number.*

> *Uniqueness enforcement: duplicate submissions are prevented at the database level via a unique constraint — @@unique([userID, userType]) — on GOVERNMENT_ID, not solely by an application-level pre-check. A pre-check (findFirst) may still run first as an optimization to avoid an unnecessary file upload in the common case, but it is not the source of truth for the 409 — the Prisma unique-constraint error (P2002) on the insert is. This closes a race condition where two near-simultaneous requests could both pass a pre-check before either had written its row. On a P2002 error, the just-uploaded Storage file is deleted before the 409 response is returned, to avoid leaving an orphaned file.*

2.2 GET /api/v1/adopters/me/government-id

Returns the current adopter's government ID verification status.

Request

Field	Value
Method	GET
Auth required	Yes — Adopter only
Path/Body fields	None — adopter identified via req.user.userID

Response

```
200 OK
{
  "success": true,
  "message": "Government ID status retrieved successfully",
  "data": {
    "idType": "Driver's License",
    "verificationStatus": "Pending",
    "submittedAt": "2026-08-01T10:15:00Z"
  }
}
```

Error Cases

Scenario	Error Code	HTTP Status
No/invalid token	UNAUTHORIZED	401
No government ID submitted yet	NOT_FOUND	404

> *Response scope: idNumber and the Storage file URL/path are never returned by this endpoint, regardless of role — only idType, verificationStatus, and submission date are exposed. This is stricter than the POST response, which masks but still includes idNumber.*

3. Adoption Applications

3.1 POST /api/v1/adoption-applications

Initiates a new adoption application for a specific pet by creating a Stripe Checkout session for the $15 application fee. The application status starts as Pending once payment succeeds; the adopter may later withdraw it themselves (see 3.5); shelter staff Accept or Reject via the same PATCH /adoption-applications/:id/status endpoint.

Request

Field	Value
Method	POST

Auth required	Yes — Adopter only
Body fields	petID (int, required), shelterID (int, required)

> *shelterID in the request body is expected to match the pet's current shelterID — it identifies which branch the application is being processed at, consistent with the ADOPTION_APPLICATION schema.*

Response

```
201 Created
{
  "success": true,
  "message": "Checkout session created",
  "data": { "checkoutUrl": "https://checkout.stripe.com/c/pay/..." }
}
```

> *Asynchronous creation: the AdoptionApplication row does not exist yet when this endpoint returns — it is created by finalizeApplication inside the Stripe webhook handler after the $15 application fee is paid. The frontend polls GET /adoption-applications?checkoutSessionId=... (see 3.2) until the row appears.*

Error Cases

Scenario	Error Code	HTTP Status
No/invalid token	UNAUTHORIZED	401
Missing petID or shelterID	VALIDATION_ERROR	422
Pet does not exist	NOT_FOUND	404
Pet adoptionStatus is not 'available'	CONFLICT	409
Adopter already has an active application for this pet	CONFLICT	409

> *Duplicate application guard: prevented at the database level via a partial unique index — CREATE UNIQUE INDEX ON ADOPTION_APPLICATION (adopterID, petID) WHERE applicationStatus IN ('Pending', 'Accepted') — not solely by an application-level pre-check. The pre-check may still run first as an early-exit optimization, but the Prisma unique-constraint error (P2002) on the insert is the actual guarantee, following the same check-then-insert-with-DB-backstop pattern used on GOVERNMENT_ID. Because the index is scoped to active statuses only, an adopter may resubmit a new application for the same pet once a prior one reaches Rejected or Withdrawn — those are terminal states, not reopened, and a resubmission creates a new applicationID rather than editing the old row.*

3.2 GET /api/v1/adoption-applications

Looks up an adoption application by its Stripe Checkout session ID. Used by the post-checkout confirmation page to poll for the row the webhook creates asynchronously (see 3.1).

Request

Field	Value
Method	GET
Auth required	Yes — Adopter (with checkoutSessionId); Staff/Admin use this endpoint as the application queue.

Query params	checkoutSessionId (string, required)

Response

```
200 OK
{
  "success": true,
  "message": "Application retrieved successfully",
  "data": {
    "applicationID": 44,
    "petID": 9,
    "adopterID": 12,
    "shelterID": 3,
    "staffID": null,
    "applicationStatus": "Pending",
    "applicationType": "...",
    "shelterMessage": "...",
    "paymentStatus": "...",
    "amountPaid": 15.00,
    "createdAt": "2026-08-03T09:00:00Z",
    "pet": { "petName": "Buddy" }
  }
}
```

> *Not-yet-ready is not an error: if the webhook hasn't finished processing the payment yet, this returns 200 with data: null, not 404 — this lets the confirmation page distinguish 'still processing' from 'invalid session'.*

Error Cases

Scenario	Error Code	HTTP Status
No/invalid token	UNAUTHORIZED	401
Missing checkoutSessionId	VALIDATION_ERROR	422

3.3 GET /api/v1/adoption-applications/:id

Returns a single adoption application by ID. Adopters may only view their own applications; staff and admins may view any application.

Request

Field	Value
Method	GET
Auth required	Yes — Adopter, Shelter Staff, Admin
Path params	id (int, required) — applicationID

> *Ownership is checked after existence: a 404 is returned before any 403 check runs, so a nonexistent applicationID never leaks whether it would have belonged to someone else. For staff, the JWT does not carry shelterID (see 06-Auth_System_Reference) — the handler re-fetches the staff member's current shelterID from the STAFF table before comparing.*

Response

```
200 OK
{
  "success": true,
  "message": "Application retrieved successfully",
  "data": {
    "applicationID": 44,
    "applicationCode": "...",
    "petID": 9,
    "adopterID": 12,
    "shelterID": 3,
    "staffID": null,
    "applicationStatus": "Pending",
    "applicationType": "...",
    "shelterMessage": "...",
    "staffRemark": "...",
    "paymentStatus": "...",
    "amountPaid": 15.00,
    "createdAt": "2026-08-03T09:00:00Z",
    "pet": { "petName": "Buddy", "petPhoto": "...", "breedName": "...", "speciesName":
"..." },
    "shelter": { "shelterName": "..." },
    "assignedStaffName": null,
    "adopter": { "...": "...contact/household fields..." },
    "governmentIdStatus": "Verified",
    "otherPendingCount": null,
    "canAssignStaff": true
  }
}
```

> *adopter, governmentIdStatus, and canAssignStaff are returned to Staff/Admin (governmentIdStatus is null for Adopters). otherPendingCount is Staff/Admin only — always null for an Adopter.*

Error Cases

Scenario	Error Code	HTTP Status
No/invalid token	UNAUTHORIZED	401
Role not adopter/staff	FORBIDDEN	403
Adopter viewing another adopter's application	FORBIDDEN	403
Staff viewing another shelter's application	FORBIDDEN	403
applicationID does not exist	NOT_FOUND	404

3.4 GET /api/v1/adopters/me/applications

Returns a paginated list of the current adopter's own adoption applications, most recent first.

Request

Field	Value
Method	GET
Auth required	Yes — Adopter only
Query params	page (int, optional, default 1), limit (int, optional, default 20), status (optional — Pending/Accepted/Rejected/Withdrawn)

> *status, when provided, filters to that single application status. Omitted, all of the adopter's applications are returned regardless of status.*

Response

```
200 OK
{
  "success": true,
  "message": "Applications retrieved successfully",
  "data": [
    {
      "applicationID": 44,
      "petID": 9,
      "shelterID": 3,
      "applicationStatus": "Pending",
      "createdAt": "2026-08-03T09:00:00Z",
      "pet": { "petName": "Buddy", "petPhoto": "...", "breed": { "breedName": "..." } }
    }
  ],
  "pagination": { "page": 1, "limit": 20, "total": 3, "totalPages": 1 }
}
```

Error Cases

Scenario	Error Code	HTTP Status
No/invalid token	UNAUTHORIZED	401
Invalid status value	VALIDATION_ERROR	422

3.5 PATCH /api/v1/adoption-applications/:id/status

Lets the adopter end their own application: Withdraw while it is Pending, or Surrender the pet once it is Accepted (approved). Both send status Withdrawn, the only adopter-initiated status transition; shelter staff Accept, Reject or Revoke via the same PATCH /adoption-applications/:id/status endpoint.

Request

Field	Value
Method	PATCH
Auth required	Yes — Adopter only (own application)
Body fields	status (string, required — only "Withdrawn" is accepted)

> *Allowed only from Pending or Accepted — these are the only non-terminal statuses. The $15 application fee is non-refundable on withdrawal; the adopter may submit a new application for the same pet afterward (see 3.1's duplicate-application note in the original doc — Withdrawn is a terminal state, not reopened).*
>
> *Surrender — from Accepted, withdrawing gives the pet back: in the same transaction the pet comes back into its shelter as 'incoming' (only if it is still 'adopted'), for staff to check before listing it again. It drops off the adopter's My Pets, and the shelter's staff get the ApplicationWithdrawn notification with the 'Pet surrendered' wording. The dashboard labels the button Surrender instead of Withdraw.*
>
> *A pet goes to one applicant. When shelter staff accept one application, the pet's other Pending applications are declined automatically (applicationStatus Rejected), with the staffRemark 'Another applicant was approved for this pet. Thank you for your interest — we hope you'll find another match.' The adopter can apply for a different pet.*

Response

```
200 OK
{
  "success": true,
  "message": "Application withdrawn successfully",
  "data": {
    "applicationID": 44,
    "applicationCode": "...",
    "petID": 9,
    "adopterID": 12,
    "shelterID": 3,
    "applicationStatus": "Withdrawn",
    "...": "same shape as GET /adoption-applications/:id (3.3)"
  }
}
```

Error Cases

Scenario	Error Code	HTTP Status
No/invalid token	UNAUTHORIZED	401
Viewing/editing another adopter's application	FORBIDDEN	403
applicationID does not exist	NOT_FOUND	404
Invalid or missing status value	VALIDATION_ERROR	422
Application not in Pending/Accepted (already Withdrawn/Rejected)	CONFLICT	409

4. Visits

Endpoints for scheduling and viewing shelter visits or virtual meet-and-greets (Requirement A-07). Distinct from ADOPTION_APPLICATION — a visit can exist independently of a formal application, and a pet or assisting staff member on a visit may be null.

4.1 POST /api/v1/visits

Schedules a shelter visit or virtual meet-and-greet for the current adopter.

Request

Field	Value
Method	POST
Auth required	Yes — Adopter only
Body fields	shelterID (int, required), petID (int, optional), visitTime (ISO 8601 datetime, required), remarks (string, optional, ≤300 chars)

> *petID is optional — a visit is not required to be tied to a specific pet (e.g. a general shelter tour). visitTime must be in the future; staffID is left null at creation and assigned later by shelter staff.*

Response

```
201 Created
{
  "success": true,
  "message": "Visit scheduled successfully",
  "data": {
    "visitID": 21,
    "adopterID": 12,
    "petID": 9,
    "staffID": null,
    "shelterID": 3,
    "visitTime": "2026-08-10T14:00:00Z",
    "remarks": "...",
    "visitStatus": null
  }
}
```

> *visitStatus stays null until shelter staff confirm the visit.*

Error Cases

Scenario	Error Code	HTTP Status
No/invalid token	UNAUTHORIZED	401
Missing shelterID or visitTime	VALIDATION_ERROR	422
visitTime is in the past	VALIDATION_ERROR	422
Shelter or pet does not exist	NOT_FOUND	404

4.2 GET /api/v1/adopters/me/visits

Returns the current adopter's scheduled visits.

Request

Field	Value
Method	GET
Auth required	Yes — Adopter only

Query params	upcoming (boolean, optional) — when true, returns only visits with visitTime in the future and visitStatus of Confirmed

Response

```
200 OK
{
  "success": true,
  "message": "Visits retrieved successfully",
  "data": [
    {
      "visitID": 21,
      "petID": 9,
      "shelterID": 3,
      "visitTime": "2026-08-10T14:00:00Z",
      "visitStatus": "Confirmed",
      "pet": { "petName": "Buddy" },
      "shelter": { "shelterName": "..." }
    }
  ]
}
```

Error Cases

Scenario	Error Code	HTTP Status
No/invalid token	UNAUTHORIZED	401

> *Not paginated: an individual adopter's visit list is expected to stay small (dozens, not hundreds), unlike the paginated shelter-wide staff queue (GET /visits).*

4.3 GET /api/v1/visits/:id

Returns a single visit by ID for the current adopter — the individual-resource counterpart to the list endpoint in 4.2.

Request

Field	Value
Method	GET
Auth required	Yes — Adopter only (own visits)
Path params	id (int, required) — visitID

Response

```
200 OK
{
  "success": true,
  "message": "Visit retrieved successfully",
  "data": {
```

```
      "visitID": 21,
      "visitTime": "2026-08-10T14:00:00Z",
      "remarks": "...",
      "visitStatus": "Confirmed",
      "canCancel": true,
      "pet": { "petName": "Buddy", "petPhoto": "...", "breedName": "...", "speciesName":
"..." },
      "shelterName": "...",
      "shelterAddress": "...",
      "assignedStaffName": null
    }
  }
```

> *canCancel is computed server-side: true only when visitStatus is not Cancelled/Completed and visitTime is still in the future — the frontend uses it to decide whether to show the Cancel action, rather than re-deriving the same logic client-side.*

Error Cases

Scenario	Error Code	HTTP Status
No/invalid token	UNAUTHORIZED	401
Viewing another adopter's visit	FORBIDDEN	403
visitID does not exist	NOT_FOUND	404

4.4 PATCH /api/v1/visits/:id

Lets the adopter cancel their own upcoming visit.

Request

Field	Value
Method	PATCH
Auth required	Yes — Adopter only (own visits)
Body fields	visitStatus (string, required — only "Cancelled" is accepted)

Response

```
200 OK
{
  "success": true,
  "message": "Visit cancelled successfully",
  "data": {
    "visitID": 21,
    "adopterID": 12,
    "petID": 9,
    "staffID": null,
    "shelterID": 3,
    "visitTime": "2026-08-10T14:00:00Z",
    "remarks": "...",
    "visitStatus": "Cancelled",
```

```
      "shelter": { "shelterName": "..." },
      "pet": { "petName": "Buddy" }
    }
  }
```

> *Staff/Admin use the same endpoint to Confirm or Complete a visit.*

Error Cases

Scenario	Error Code	HTTP Status
No/invalid token	UNAUTHORIZED	401
Cancelling another adopter's visit	FORBIDDEN	403
visitID does not exist	NOT_FOUND	404
Invalid or missing visitStatus	VALIDATION_ERROR	422
Visit already in the past, already Cancelled, or already Completed	CONFLICT	409

5. Appointments

Read-only endpoints for the adopter to view their pet's vet appointments.

5.1 GET /api/v1/adopters/me/appointments

Request

Field	Value
Method	GET
Auth required	Yes — Adopter only
Query params	upcoming (boolean, optional — only the exact string "true" filters to future appointments)

Response

```
200 OK
{
  "success": true,
  "message": "Appointments retrieved successfully",
  "data": [
    {
      "appointmentID": 8,
      "appointmentDate": "2026-09-01T10:00:00Z",
      "appointmentReason": "Annual checkup",
      "pet": { "petID": 9, "petName": "Buddy" },
      "shelter": { "shelterName": "..." },
      "vet": { "vetName": "..." }
    }
  ]
}
```

> *Not paginated, same rationale as Visits (4.2) — an individual adopter's appointment list stays small. Only appointments for pets the adopter has an Accepted application for are returned.*

Error Cases

Scenario	Error Code	HTTP Status
No/invalid token	UNAUTHORIZED	401

5.2 GET /api/v1/adopters/me/appointments/:id

Request

Field	Value
Method	GET
Auth required	Yes — Adopter only
Path params	id (int, required) — appointmentID

Response

```
200 OK
{
  "success": true,
  "message": "Appointment retrieved successfully",
  "data": {
    "appointmentID": 8,
    "appointmentCode": "...",
    "appointmentDate": "2026-09-01T10:00:00Z",
    "appointmentReason": "Annual checkup",
    "pet": { "petID": 9, "petName": "Buddy", "petPhoto": "...", "breedName": "...",
"speciesName": "..." },
    "vetName": "...",
    "shelterName": "...",
    "shelterAddress": "...",
    "vaccinesAdministered": [
      { "recordID": 15, "vaccineName": "Rabies", "dueDate": "2027-01-10T00:00:00Z" }
    ]
  }
}
```

> *vaccinesAdministered lists the VACCINATION_RECORD rows linked to this appointment by appointmentID.*

Error Cases

Scenario	Error Code	HTTP Status
No/invalid token	UNAUTHORIZED	401
No Accepted application links this adopter to the appointment's pet	NOT_FOUND	404

6. Favorites

Endpoints for saving and removing pets from the adopter's favorites list (Requirement A-04). Backed by the FAVORITES junction table (adopterID, petID).

6.1 POST /api/v1/pets/:id/favorites

Adds a pet to the current adopter's favorites list.

Request

Field	Value
Method	POST
Auth required	Yes — Adopter only
Path params	id (int, required) — petID

Response

```
201 Created
{
  "success": true,
  "message": "Pet added to favorites",
  "data": { "adopterID": 12, "petID": 9 }
}
```

Error Cases

Scenario	Error Code	HTTP Status
No/invalid token	UNAUTHORIZED	401
Pet does not exist	NOT_FOUND	404
Pet already in favorites	CONFLICT	409

> *Uniqueness: FAVORITES has a composite primary key on (adopterID, petID) — the same check-then-insert-with-DB-backstop pattern applies here for free, since a composite PK is itself a uniqueness constraint. A duplicate insert throws P2002, caught and returned as 409.*

6.2 DELETE /api/v1/pets/:id/favorites

Removes a pet from the current adopter's favorites list.

Request

Field	Value
Method	DELETE
Auth required	Yes — Adopter only
Path params	id (int, required) — petID

Response

```
200 OK
{
  "success": true,
  "message": "Pet removed from favorites",
  "data": null
}
```

Error Cases

Scenario	Error Code	HTTP Status
No/invalid token	UNAUTHORIZED	401
Pet not in favorites (nothing to remove)	NOT_FOUND	404

6.3 GET /api/v1/adopters/me/favorites

Returns the current adopter's full list of favorited pets.

Request

Field	Value
Method	GET
Auth required	Yes — Adopter only
Path/Body fields	None — adopter identified via req.user.userID

Response

```
200 OK
{
  "success": true,
  "message": "Favorites retrieved successfully",
  "data": [
    {
      "petID": 9,
      "petName": "Buddy",
      "petAge": "...",
      "petSex": "M",
      "petPhoto": "...",
      "petColor": "...",
      "petHeight": 45.0,
      "petWeight": 12.4,
      "petDesc": "...",
      "breed": { "breedID": 3, "breedName": "...", "speciesName": "..." },
      "shelter": { "shelterID": 3, "shelterName": "...", "shelterAddress": "..." },
      "compatibleWithChildren": true,
      "compatibleWithPets": true,
      "specialNeeds": false
    }
```

```
    ]
}
```

Error Cases

Scenario	Error Code	HTTP Status
No/invalid token	UNAUTHORIZED	401

> *Response shape mirrors the full public pet-detail response of GET /pets/:id (see 07-Public API Reference), not the catalog card shape — each favorite carries the same fields as the public pet detail page.*

7. Adopted Pets

Endpoints for an adopter to view pets they have adopted or fostered, including each pet's universal health passport (medical history and vaccination records that persist across inter-shelter transfers).

7.1 GET /api/v1/adopters/me/adopted-pets

Returns the pets currently or previously adopted/fostered by the current adopter, identified via ADOPTION_APPLICATION records with applicationStatus = 'Accepted' joined to PET.

Request

Field	Value
Method	GET
Auth required	Yes — Adopter only
Path/Body fields	None — adopter identified via req.user.userID

Response

```
200 OK
{
  "success": true,
  "message": "Adopted pets retrieved successfully",
  "data": [
    {
      "petID": 9,
      "petName": "Buddy",
      "petAge": "...",
      "petSex": "M",
      "petPhoto": "...",
      "breed": { "breedName": "...", "speciesName": "..." }
    }
  ]
}
```

Error Cases

Scenario	Error Code	HTTP Status
No/invalid token	UNAUTHORIZED	401

> *Slim PetCard shape: this list intentionally mirrors the same summary shape used by Favorites (6.3) and the public catalog card, for rendering with the same PetCatalogCard/list component. The full profile — including health passport data — is available per-pet via GET /adopters/me/adopted-pets/:petId (7.2).*

7.2 GET /api/v1/adopters/me/adopted-pets/:petId

Returns the full consolidated profile for one adopted/fostered pet, including grouped health-passport and adoption-history data. This is a richer, adopter-specific shape — not the same as the public GET /pets/:id response.

Request

Field	Value
Method	GET
Auth required	Yes — Adopter only (must have an Accepted application for this pet)
Path params	petId (int, required)

Response

```
200 OK
{
  "success": true,
  "message": "Pet profile retrieved successfully",
  "data": {
    "petID": 9,
    "petCode": "...",
    "petName": "Buddy",
    "petPhoto": "...",
    "microchipID": "982000123456789",
    "petAge": "2 years, 7 months",
    "petDOB": "2023-02-14",
    "petSex": "M",
    "petColor": "Brown",
    "petSize": "Medium",
    "petHeight": 45.0,
    "petWeight": 12.4,
    "petBGroup": "A",
    "petDesc": "...",
    "adoptionStatus": "adopted",
    "breed": { "breedName": "...", "speciesName": "..." },
    "compatibility": { "children": true, "otherPets": true, "specialNeeds": false },
    "health": {
      "vaccinations": [
        { "recordID": 15, "vaccineName": "Rabies", "administeredDate": "2026-01-
10T00:00:00Z", "dueDate": "2027-01-10T00:00:00Z", "vetName": "..." }
      ],
```

```
      "appointments": [
        { "appointmentID": 8, "appointmentDate": "2026-09-01T10:00:00Z",
"appointmentReason": "Annual checkup", "vetName": "...", "shelterName": "..." }
      ]
    },
    "adoption": { "adoptedOn": "2026-06-01", "shelterName": "...", "shelterAddress":
"...", "yourMessage": "...", "staffRemark": "..." }
  }
}
```

> *health.appointments lists upcoming appointments only. This endpoint is the closest thing to the 'universal health passport' view described in the section intro — it groups medical/vaccination data that a future inter-shelter transfer feature would carry with the pet.*

Error Cases

Scenario	Error Code	HTTP Status
No/invalid token	UNAUTHORIZED	401
Adopter has no Accepted application for this pet	FORBIDDEN	403
petId does not exist	NOT_FOUND	404

7.3 GET /api/v1/adopters/me/adopted-pets/:petId/vaccinations

Returns the vaccination history for one of the current adopter's adopted/fostered pets — part of the pet's universal health passport.

Request

Field	Value
Method	GET
Auth required	Yes — Adopter only
Path params	petId (int, required)

> *Ownership check: the handler verifies an Accepted ADOPTION_APPLICATION exists linking this adopter to this pet before returning any vaccination data — an adopter cannot view vaccination records for a pet they have not adopted, even if they know the petId.*

Response

```
200 OK
{
  "success": true,
  "message": "Vaccination records retrieved successfully",
  "data": [
    {
      "recordID": 15,
      "petID": 9,
      "vaccineID": 2,
      "vaccine": { "vaccineName": "Rabies", "manufacturer": "..." },
```

```
        "administeredDate": "2026-01-10T00:00:00Z",
        "dueDate": "2027-01-10T00:00:00Z",
        "administeredBy": 5,
        "administeredAt": 3
      }
    ]
  }
```

Error Cases

Scenario	Error Code	HTTP Status
No/invalid token	UNAUTHORIZED	401
Pet does not exist	NOT_FOUND	404
Adopter has not adopted this pet	FORBIDDEN	403

> *Response shape matches the VACCINATION_RECORD table directly, joined to VACCINE for display fields (vaccineName, manufacturer) — this is the same shape a future vet-facing endpoint would use to write these records, kept consistent rather than reshaping per-consumer.*

> *This is read-only for adopters. Recording or updating a vaccination is restricted to the Vet role (recorded through the Vet portal), consistent with NF-02 RBAC.*

8. AI matcher (Sprint 8)

Endpoints for the adopter personality quiz and compatibility matches. All four are Adopter-only and require a Bearer access token.

GET /api/v1/match-quiz/questions — the personality quiz questions and their options.

GET /api/v1/adopters/me/quiz — the adopter's saved quiz answers (null if not taken).

PUT /api/v1/adopters/me/quiz — save or retake the whole quiz; clears the adopter's cached matches.

GET /api/v1/adopters/me/matches — the adopter's top compatibility matches.

Full request/response shapes, scoring, caching and error cases are in 15-AI Features API Reference.